

Programming and Databases

Joern Ploennigs

Exceptions

MIDJOURNEY: EXCEPTION IN TIME, REF. SALVADOR DALÍ

RECAP: LECTURE HALL QUESTION

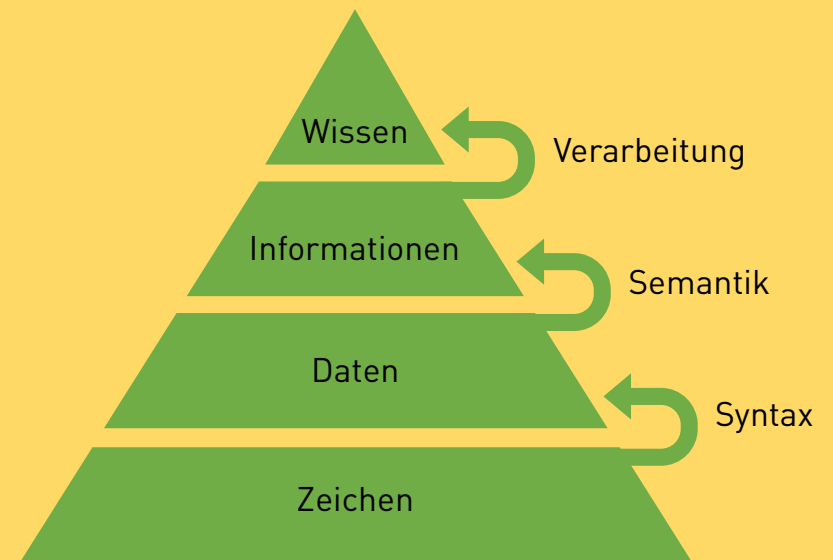
What levels does the knowledge pyramid have?



Midjourney: Tower of Babel

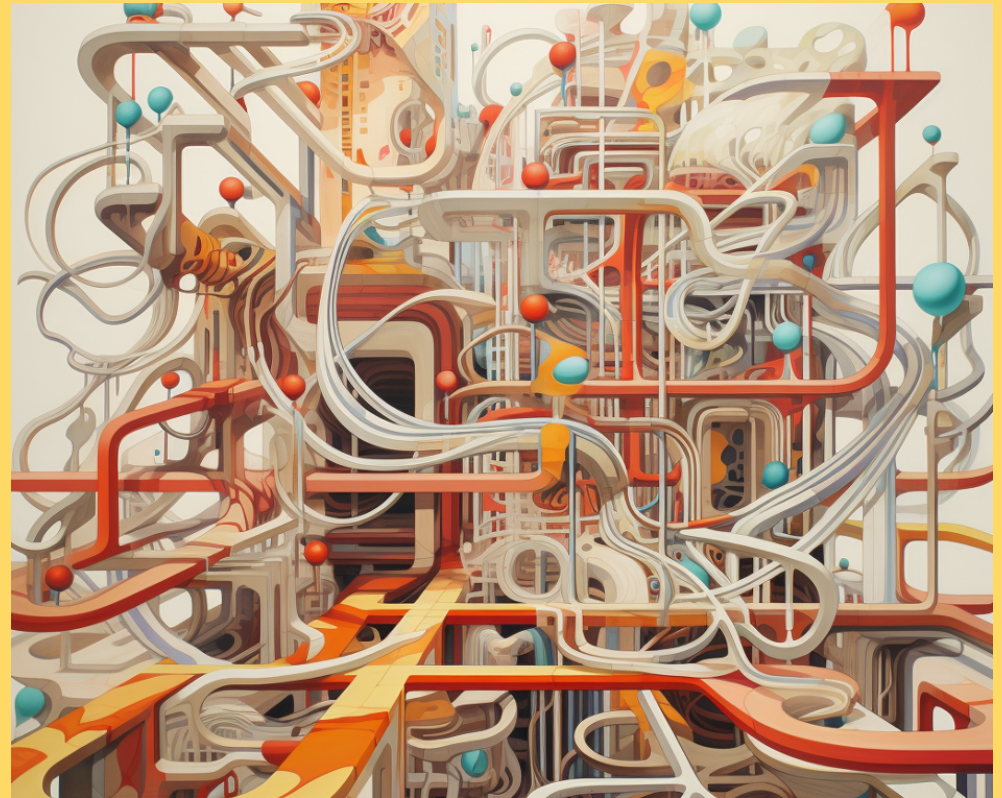
REVIEW: KNOWLEDGE PYRAMID

The Wissenspyramide is a model for representing the emergence of knowledge. The four element types: *Signs*, *Data*, *Information* and *Knowledge* are shown pyramid-shaped as four levels. Signs form the base and *Knowledge* the tip of the pyramid.



RECAP: LECTURE HALL QUESTION

What are the four fundamental elements of a program?

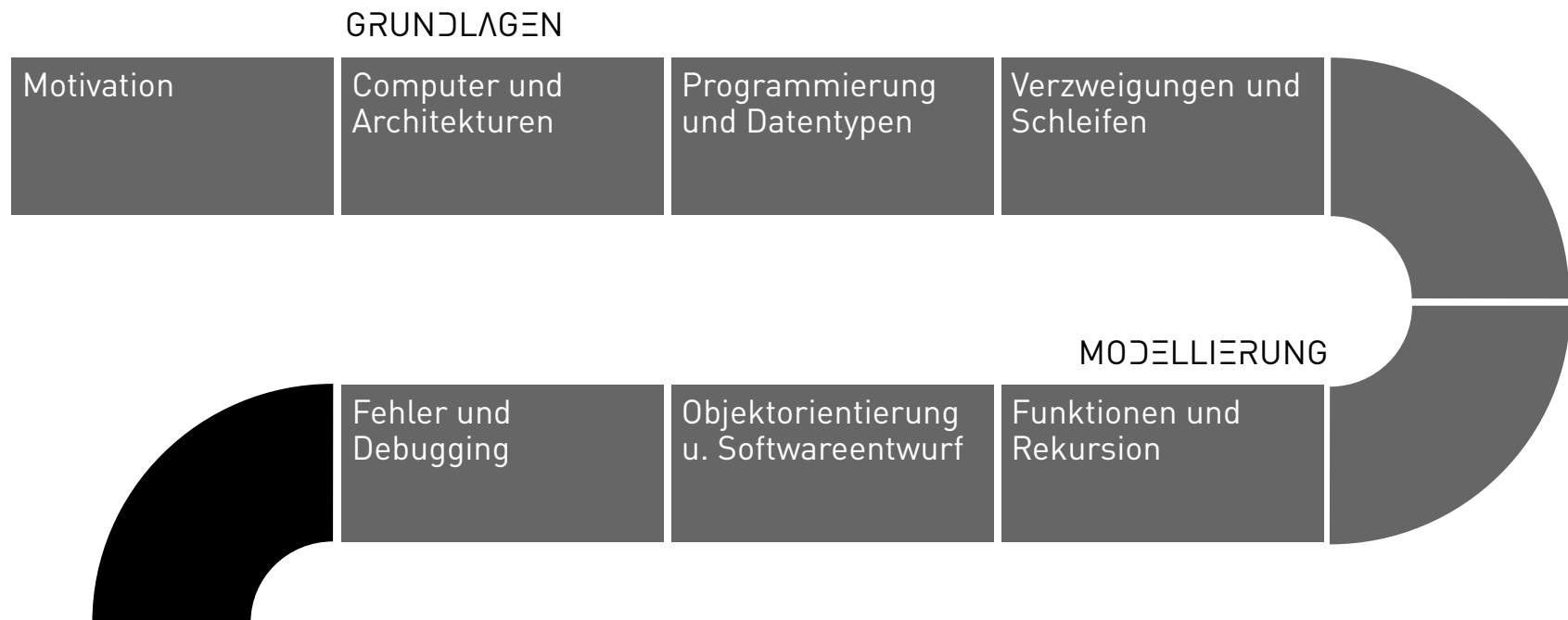


Midjourney: Fundamentals of a Programming Language

RECAP: PROGRAM ELEMENTS

- We all know the basic building blocks of a program:
 - Statements
 - Functions
 - Branching
 - Loops & Recursion
- We know how to plan algorithms and combine these elements.

PROCESS



WHY DO WE MAKE MISTAKES?

Deliberate mistakes occur when:

- we consciously ignore facts and trust our gut instincts
- when we run tests to test whether a program handles these errors
- through targeted sabotage (insider threat in information security)

Unintentional mistakes occur, among other things, due to:

- false assumptions, for example, because the requirements were not analyzed sufficiently or because not all error cases were considered
- inattention (careless mistakes) due to fatigue, time pressure, or lack of interest
- because we reach cognitive/physical limits (e.g., optical illusions)

MISTAKES ARE HUMAN

- *Computers are human-made*
- The mental models of others have (often subtle) differences from our own, e.g. due to:
 - Age
 - Background
 - Education
 - Experience
 - Culture
- Most everyday mistakes happen at the *interface between us and the internal processes*: user interfaces, hardware, etc.

LECTURE HALL QUESTION

What does this symbol mean?



icon-icons.com

WHAT DOES THIS SYMBOL MEAN?

- The floppy disk icon is commonly used to save.
- It originally comes from the 1990s when data was still stored on floppy disks.
- Nowadays many people don't have this context and can't intuitively interpret the symbol.
- An example of different mental models across generations



icon-icons.com



www.pxfuel.com

WHEN DO COMPUTERS MAKE MISTAKES?

- Computers very rarely make errors (e.g., bit flips due to cosmic rays, errors in floating-point calculations, integer overflows, errors in data transmission or storage)
- Most of the time, however, it is the user or programmer who made the errors possible (wrong data types, poor exception or error handling)
- The real question is: Are there errors in the system beyond our own program code?

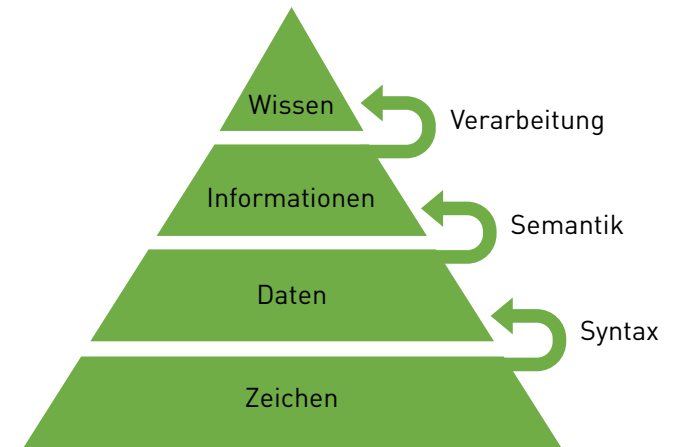
HOW TO PREVENT ERRORS?

- *Using an Integrated Development Environment (IDE)*
 - Automatic detection of syntax errors
 - Code completion
 - Code generation (e.g., from diagrams)
- *Comprehensive planning of the software, including requirements analysis, software design, test design, etc.*
- *TESTING, TESTING, TESTING* using specific test cases (unit tests)!
 - Not just pure functional testing (does it do what it's supposed to?)

TYPES OF ERRORS

Along the Knowledge Pyramid:

- *Lexical* – Code contains invalid characters or strings
- *Syntactic* – Code combines lexically valid tokens in an invalid way
- *Semantic* – Code is semantically valid, but does not produce any results/crashes
- *Logical* – Code is semantically valid, but does not solve the given problem



OTHER CLASSIFICATIONS OF ERRORS

By frequency and reproducibility:

- *Deterministic errors* - occur every time and can be reproduced
- *Sporadic errors* - occur only rarely and under special conditions. They are difficult to reproduce

By occurrence:

- *Static errors* - Directly observable in the code as written
- *Dynamic errors* - Only occur during program execution

LEXICAL & SYNTACTIC ERRORS

- Are in most cases *static* and *deterministic*
- Produce an error message *every time*
 - Even in the IDE, during compilation or after execution
- *These error messages can, however, be misleading!*

LEXICAL & SYNTACTIC ERRORS - EXAMPLES

- A *keyword* is misspelled
- *Unclosed* quotation marks or parentheses
- Forgetting a *colon* after a condition or function
- *Incorrect indentation* after a condition, loop, or function
- If applicable, also incorrect indentation due to *tabs vs. spaces*

LEXICAL & SYNTACTIC ERRORS - HANDLING

- *First look*: In which line did the error occur?
- Does the *text of the error message* give a hint?
- Is the error in the line, or is the error only a *symptom* and the real error lies in the previous line?
- Normally *easy to fix* and go through the code line by line backwards from the error line
- If not: *Refactoring* - restructure the code for better readability without changing its behavior

SEMANTIC ERRORS - FUNDAMENTALS

- Arise from the *faulty application* of syntactically correct program structures to a specific problem
- Semantic errors can be *recognizable* or *unrecognizable*. They are always deterministic, but can occur statically or dynamically
- *Recognizable errors* are errors of which one knows that they can occur (e.g., division by zero, no Internet connection, etc.)
 - They should *always* be caught and handled
- *Unrecognized errors* are unknown during the creation of the program. Significantly harder to catch

SEMANTIC ERRORS - EXAMPLES

- *An variable is invoked as a function*
(static, deterministic)
- *An operator is used with a value of the wrong type*
(static or dynamic, deterministic)
- *An value of the wrong type is passed to a function*
(static or dynamic, deterministic)
- *A division by zero occurs*
(dynamic, deterministic)

SEMANTIC ERRORS - HANDLING

First step: Understanding the error message

- *Ideal case:* Indicates the exact problem
- *Usually:* Provides clues about a problem
 - Location
 - Type of problem
 - The affected variable/operation
- *Often:* The error message is only a symptom of the underlying problem

SEMANTIC ERRORS - HANDLING (2)

If the error message does not provide enough information:

- *Log the program flow* to understand it better:
 - Add more output (e.g., via the `print()` function)
 - Check variable states or verify whether certain lines are reached
- *Use debugging tools:*
 - Pause the program flow at certain points (*Breakpoints*)
 - Step through the program flow
 - *Watch variables* at that moment

LOGICAL ERRORS – FUNDAMENTALS

Arise from:

- Errors in the *program logic*
- A *faulty mental model*
- A fundamental *design flaw*
- *Careless mistakes* in translating from design to implementation

Characteristics:

- They *rarely produce exceptions* but rather incorrect behavior/results
- They often go *undetected* and only manifest dynamically at runtime
- They can be *deterministic* or *sporadic*

LOGICAL ERRORS - EXAMPLES

Deterministische Fehler:

- Using the *wrong factor* in a unit conversion
- Using the *wrong type of loop* or incorrect nesting
- Using the *wrong time zone* when writing sensor data

Sporadische Fehler:

- Issues with *Daylight Saving Time conversions* for sensor data
- The condition in a loop has cases in which it is *always true* and thus becomes endless

HANDLING OF LOGICAL ERRORS

Deterministische Fehler:

- *Create test cases* to reproduce when and how unexpected behavior occurs
- *Use the debugger* to trace the execution and isolate the root cause of the error
- Track values *systematically* and verify correctness step by step

Sporadische Fehler:

- Identify the error state via *Logging* (`print()`) so that it becomes reproducible
- *Conditional breakpoints* in debugging (the program only stops when an error condition is present)

NOTE: WARNINGS IN THE IDE OR FROM THE COMPILER

- Warnings point to code that *appears syntactically and semantically correct*, but will probably not fulfill its purpose
- They are based, for example, on *empirical evidence*
- They can help *detect semantic and logical errors early*

LESSON LEARNED

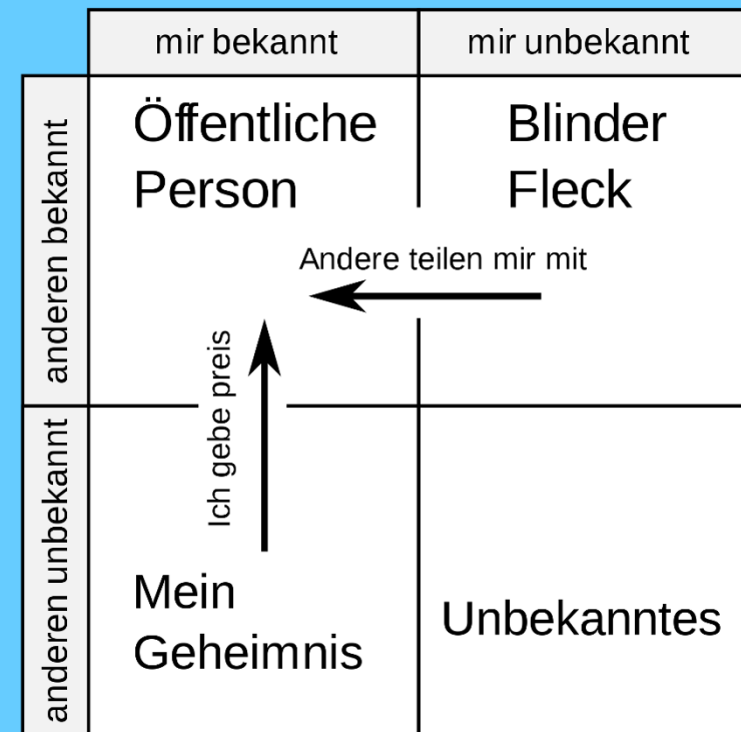
What is feedback good for?



Midjourney: Dorian Gray looking at his picture seeing an old monster

LESSON LEARNED - SELF AND OTHERS' PERCEPTION

- Our self-image never quite matches the image others have of us, because we bring our experiences and thoughts into it
- The image others have of us is formed solely through observation and descriptions of us
- Feedback is crucial for aligning self-image and the image others have of us



Questions?

